

La implementación en Python, línea por ecuación

Inteligencia Computacional · FICH-UNL

El código del TP2 mapeado contra las ecuaciones de los apuntes 02 y 04

Índice

0. Para qué es este apunte	1
1. Las tres decisiones de diseño	1
2. La clase completa	3
3. El mapa: cada ecuación, su línea	4
4. El recorrido de un patrón	5
5. Verificación sobre el XOR	6
6. Preguntas probables, y por dónde arrancar la respuesta	7
7. Ejercicio 2: las clases concéntricas	8
8. Lo que todavía no está	11

0. Para qué es este apunte

Los apuntes 02 y 04 deducen las ecuaciones. Éste toma **el código que ya está en el TP2** y muestra qué línea es cada ecuación, para que en la defensa puedas señalar el renglón y decir la fórmula.

No es una implementación nueva: es **la clase `PerceptronMulticapa` del notebook**, documentada.

Referencias cruzadas: las ecuaciones citadas están en 04-perceptron-multicapa.md; las deducciones para rehacerlas en la pizarra, en 05-derivaciones-para-pizarra.md.

1. Las tres decisiones de diseño

Antes del código, las tres decisiones que hay que poder justificar. Son lo primero que se pregunta.

1.1 Una matriz por capa, guardadas en una lista

$W^I, W^{II}, W^{III} \rightarrow \text{self.pesos} = [W1, W2, W3]$

Así el algoritmo no depende de cuántas capas haya: los bucles recorren la lista. Es la traducción directa de la fórmula general del apunte 04 §13, que vale para una capa p cualquiera.

Con `neuronas_por_capa=[2, 4, 1]` quedan:

capa 1: pesos (2, 4), umbrales (4,)

capa 2: pesos (4, 1), umbrales (1,)

Notá el orden de los índices: **`pesos[capa][i, j]` es el peso de la neurona i de la capa anterior hacia la neurona j de la capa actual**. Es la traspuesta de la convención w_{ji} del apunte, y eso es a propósito: permite escribir la propagación como `activacion @ pesos`, que es lo natural en NumPy.

1.2 El sesgo va aparte, en umbrales

En el apunte el sesgo entra como una entrada más, $x_0 = -1$ con su peso w_{j0} . En el código está separado y **sumando**:

apunte: $v_j = \sum_{i=1}^N w_{ji}x_i - w_{j0}$ código: `entrada_neta = activacion @ pesos + umbral`

PARA LA DEFENSA — son lo mismo, con el signo cambiado

Es la pregunta más probable sobre el código. Las dos formas son **equivalentes**, y la relación es

$$b_j = -w_{j0}$$

porque en el apunte el término del sesgo es $w_{j0} \cdot x_0 = w_{j0} \cdot (-1) = -w_{j0}$. Ninguna es más correcta: la entrada extendida es más compacta para deducir (un solo producto interno), y el vector aparte es más cómodo para programar (no hay que agregarle una columna de -1 a los datos en cada capa). **Lo que no se puede hacer es mezclarlas y perder el signo.**

1.3 La derivada se calcula con la salida, no con la entrada neta

```
def derivada_activacion(self, salida):  
    return 0.5 * (1 - salida ** 2)
```

Es exactamente $\varphi'(v_j) = \frac{1}{2}(1 + y_j)(1 - y_j)$ del apunte 04 §9, factorizado como diferencia de cuadrados: $(1 + y)(1 - y) = 1 - y^2$.

Por eso el parámetro se llama `salida` y no `entrada_neta`: **no hace falta guardar los v ni recalcul ninguna exponencial**, alcanza con las activaciones que la propagación hacia adelante ya dejó guardadas.

Claves de la sección 1

Clave	Qué tenés que poder responder
Lista de matrices	Por qué el algoritmo no depende de la cantidad de capas
Orden de índices	Por qué pesos[capa][i, j] está traspuesto respecto de w_{ji}
El sesgo	La relación $b_j = -w_{j0}$ y por qué las dos formas son equivalentes
La derivada	Por qué recibe la salida y no v

2. La clase completa

```

class PerceptronMulticapa:
    def __init__(self, neuronas_por_capa, tasa_aprendizaje=0.1, semilla=None):
        generador = np.random.default_rng(semilla)
        self.tasa_aprendizaje = tasa_aprendizaje
        self.pesos = [
            generador.uniform(-0.5, 0.5, (neuronas_por_capa[i], neuronas_por_capa[i + 1]))
            for i in range(len(neuronas_por_capa) - 1)
        ]
        self.umbrales = [
            generador.uniform(-0.5, 0.5, neuronas_por_capa[i + 1])
            for i in range(len(neuronas_por_capa) - 1)
        ]

    def activacion(self, entrada_neta):
        return 2 / (1 + np.exp(-entrada_neta)) - 1

    def derivada_activacion(self, salida):
        return 0.5 * (1 - salida ** 2)

    def propagar(self, entradas):
        activaciones = [entradas]
        for pesos_capa, umbral_capa in zip(self.pesos, self.umbrales):
            entrada_neta = activaciones[-1] @ pesos_capa + umbral_capa
            activaciones.append(self.activacion(entrada_neta))
        return activaciones

    def entrenar_un_patron(self, entradas, salida_deseada):
        activaciones = self.propagar(entradas)
        error_salida = salida_deseada - activaciones[-1]

        deltas = [error_salida * self.derivada_activacion(activaciones[-1])]
        for capa in range(len(self.pesos) - 1, 0, -1):

```

```

delta_propagado = self.derivada_activacion(activaciones[capa]) * (self.pesos[capa]
deltas.insert(0, delta_propagado)

for capa in range(len(self.pesos)):
    self.pesos[capa] = self.pesos[capa] + self.tasa_aprendizaje * np.outer(activacione
    self.umbrales[capa] = self.umbrales[capa] + self.tasa_aprendizaje * deltas[capa]

return error_salida

```

(el resto —entrenar, probar, _copiar_estado— es contabilidad de épocas e historial, no algoritmo)

3. El mapa: cada ecuación, su línea

Ésta es la tabla para tener a mano en la defensa.

Apunte 04	Ecuación	Línea de código
§2	$\mathbf{W}^{(p)}$, una matriz por capa	self.pesos = [...]
§4	$\varphi(v) = \frac{2}{1+e^{-v}} - 1$	2 / (1 + np.exp(-entrada_neta)) - 1
§9	$\varphi'(v_j) = \frac{1}{2}(1 + y_j)(1 - y_j)$	0.5 * (1 - salida ** 2)
§3	$v_j^{(p)} = \langle \mathbf{w}_j^{(p)}, \mathbf{y}^{(p-1)} \rangle$	activaciones[-1] @ pesos_capa + umbral_capa
§3	$y_j^{(p)} = \varphi(v_j^{(p)})$	activaciones.append(self.activacion(..
§5	$e_j = d_j - y_j^{III}$	error_salida = salida_deseada - activaciones[-1]
§11	$\delta_j^{III} = e_j \varphi'(y_j^{III})$	error_salida * self.derivada_activacion(activaciones[
§12	$\delta_j^{(p)} =$ $[\sum_k \delta_k^{(p+1)} w_{kj}^{(p+1)}] \varphi'(y_j^{(p)})$	self.derivada_activacion(...) * (self.pesos[capa] @ deltas[0])
§10	$\Delta w_{ji} = \mu \delta_j y_i$	self.tasa_aprendizaje * np.outer(activaciones[capa], deltas[capa])
§14	$w(n+1) = w(n) + \Delta w$	self.pesos[capa] = self.pesos[capa] + ...
§14	$\Delta w_{j0} = \mu \delta_j (-1)$	self.umbrales[capa] + self.tasa_aprendizaje * deltas[capa]

OJO — la línea del sesgo no tiene el (-1)

En el apunte el ajuste del peso de sesgo es $\mu \delta_j (-1)$, y en el código es $+ \text{tasa} * \text{deltas}[\text{capa}]$, sin el menos. **No es un error:** como $b_j = -w_{j0}$, al derivar respecto de b en vez de w_0 el signo se da vuelta y

queda $\Delta b_j = +\mu \delta_j$. Los dos ajustan la frontera para el mismo lado. Si te lo preguntan, la respuesta corta es: “el código optimiza b , no w_0 , y $b = -w_0$ ”.

PARA LA DEFENSA — dónde señalar cuando pregunten “¿dónde está la retropropagación?”

En `self.pesos[capa] @ deltas[0]`. Eso es el corchete $\sum_k \delta_k w_{kj}$: el producto de la **matriz de pesos** por el **vector de deltas de la capa siguiente**. Es el paso que hace pasar el error por los mismos pesos, en sentido contrario. Y el bucle va `range(len(self.pesos) - 1, 0, -1)` —**al revés**— porque el δ de una capa necesita el de la capa siguiente ya calculado.

4. El recorrido de un patrón

Los tres pasos del apunte 04 §14, en el orden en que los hace el código:

1. **Hacia adelante** (propagar). Devuelve una **lista con las activaciones de todas las capas**, no sólo la salida. Eso no es un capricho: los pasos 2 y 3 las necesitan.
2. **Hacia atrás**. Se calcula primero δ de la salida, y después el bucle va de la anteúltima capa hacia la primera insertando cada delta al principio de la lista (`deltas.insert(0, ...)`), para que al final el índice de `deltas` coincida con el de pesos.
3. **Ajuste**. Recién acá se tocan los pesos, en un bucle aparte.

PARA LA DEFENSA — por qué el ajuste está en un bucle separado

Es el punto fino de la clase (apunte 04 §14). Las activaciones y los deltas **ya están calculados con los pesos viejos**, así que el bucle de ajuste puede recorrer las capas en cualquier orden y da lo mismo. Lo que **no** se puede hacer es intercalar: si actualizaras `pesos[capa]` dentro del bucle de deltas, el delta siguiente se calcularía con pesos a medio mover y el gradiente dejaría de ser el gradiente en $w(n)$. Por eso son dos bucles y no uno.

IDEA DE FONDO — es entrenamiento estocástico

`entrenar_un_patron` ajusta los pesos **después de cada patrón**, no al final de la época. Eso es coherente con el criterio de error **instantáneo** $\xi(n)$ del apunte 04 §5: nunca se promedia sobre el conjunto. Es lo que justifica usar μ chico.

Claves de la sección 4

Clave	Qué tenés que poder responder
Qué devuelve propagar	Por qué la lista completa y no sólo la salida
El bucle al revés	Por qué los deltas se calculan de atrás para adelante
Dos bucles	Por qué el ajuste está separado del cálculo de deltas

5. Verificación sobre el XOR

Corrí la clase con los datasets del TP2, variando la cantidad de neuronas ocultas y la semilla. Ojo con lo que sale:

```

--- 2 neuronas ocultas: convergen 8/10
semilla    0: converge      4 epocas   100.0% test
semilla    1: converge      4 epocas   100.0% test
semilla    2: converge      4 epocas   100.0% test
semilla    3: converge      4 epocas   100.0% test
semilla    7: converge      4 epocas   100.0% test
semilla   11: converge      4 epocas   100.0% test
semilla   42: NO conv.     300 epocas    56.0% test
semilla   99: NO conv.     300 epocas    56.0% test
semilla  123: converge      4 epocas   100.0% test
semilla 2024: converge      4 epocas   100.0% test

```

```

--- 4 neuronas ocultas: convergen 10/10   (3 a 5 épocas, 100% en todas)

```

OJO — cuidado con cómo se enuncia esto en la defensa

El notebook compara “XOR con 4 ocultas” contra “XOR con 2 ocultas” y muestra que la segunda no converge. **Con la semilla 42 es cierto — pero es un resultado de esa semilla, no de la arquitectura.** Con 8 de 10 semillas, dos neuronas ocultas convergen en 4 épocas y clasifican 100%. Si decís “con 2 ocultas no converge” y el profe prueba con otra semilla, quedás pagando. La afirmación defendible es: “*con dos neuronas ocultas la convergencia depende de la inicialización: en algunas corridas cae en un mínimo local y se queda en el 56%, que es nivel de azar. Con cuatro, converge siempre.*”

PARA LA DEFENSA — esto conecta con la teoría, y vale oro

Es el recuadro de *existencia contra aprendizaje* del apunte 04 §1, ahora con evidencia propia: - En el apunte 03 **diseñamos a mano** una red de dos neuronas ocultas que resuelve el XOR. O sea que la solución con dos **existe**. - Acá se ve que el algoritmo, desde una inicialización al azar, **a veces no la encuentra**.

Ésa es exactamente la letra chica de “tres capas resuelven cualquier problema”: la arquitectura lo permite, encontrar los pesos es otra cosa. Y agregar neuronas de más no cambia lo que la red *puede* representar — **cambia la probabilidad de que el entrenamiento llegue**.

OJO — qué mide tolerancia

El criterio de corte cuenta un patrón como fallado si $|e| > 0,1$ sobre la **salida continua**, no si se equivoca de clase. Es **más estricto** que el error de clasificación: una salida de 0,7 cuando se esperaba 1 clasifica bien pero cuenta como error. Por eso errores_por_epoca puede no dar cero aunque el porcentaje de aciertos sea 100%.

Claves de la sección 5

Clave	Qué tenés que poder responder
2 vs. 4 ocultas	Qué se puede afirmar y qué no
Mínimos locales	Por qué la misma arquitectura converge o no según la semilla
Existencia vs. aprendizaje	Cómo se conecta con el diseño a mano del apunte 03
tolerancia	Qué mide, y por qué no es el error de clasificación

6. Preguntas probables, y por dónde arrancar la respuesta

Si te preguntan...	Señalá	Y decí
"¿Dónde está el sesgo?"	<code>self.umbrales</code>	que es la entrada $x_0 = -1$ del apunte, con $b = -w_0$
"¿Por qué esa derivada?"	<code>0.5 * (1 - salida ** 2)</code>	que es $\frac{1}{2}(1+y)(1-y)$ factorizado, y que depende de la salida para no recalcular la exponencial
"¿Dónde está la retropropagación?"	<code>self.pesos[capa] @ deltas[0]</code>	que es $\sum_k \delta_k w_{kj}$: el error pasando por los mismos pesos, al revés
"¿Por qué guardás todas las activaciones?"	<code>return activaciones</code>	que el ajuste usa las salidas calculadas con los pesos viejos
"¿Por qué dos bucles y no uno?"	<code>for capa in ...</code>	que primero van todos los deltas y después todos los ajustes
"¿Actualiza por patrón o por época?"	<code>entrenar_un_patron</code>	por patrón: es estocástico, coherente con $\xi(n)$ instantáneo
"¿Por qué con 2 ocultas a veces falla?"	la tabla de semillas	mínimo local; existencia $\neq$ aprendizaje
"¿Cuándo corta el entrenamiento?"	<code>if errores_en_esta_epoca == 0: break</code>	y aclarar qué cuenta como error (la tolerancia, no la clase)

7. Ejercicio 2: las clases concéntricas

La consigna pide **determinar la estructura apropiada y representar gráficamente la clasificación**. La estructura no se busca a los tumbos: se argumenta con los tres pasos del apunte 04 §1.

7.1 Qué dicen los datos

```
concent_trn: 1499 patrones, 2 entradas    concent_tst: 1000 patrones
centroide: (0.501, 0.496)
clase -1: 552 patrones, radio 0.013 a 0.303    <- el disco interior
clase +1: 947 patrones, radio 0.297 a 0.503    <- el anillo exterior
```

Las dos clases se separan casi perfectamente con **una circunferencia de radio 0,30 centrada en (0,5; 0,5)**: ese círculo ideal clasifica bien el 99,5% del entrenamiento. Ésa es la cota superior contra la que hay que comparar la red.

7.2 El argumento de la arquitectura

1. **Forma necesaria:** la clase -1 está **encerrada** por la $+1$, así que hace falta una región **cerrada**.
2. **Cuántas capas:** un recinto cerrado aproximadamente circular es **convexo**, y las regiones convexas cerradas son lo que dan **dos capas**. No hay concavidades ni huecos, así que la tercera capa no aporta nada.
3. **Cuántas neuronas:** N ocultas dan un polígono de hasta N lados. Con 2 **no se puede cerrar**; con 3 apenas un triángulo; a más neuronas, mejor aproximación al círculo.

7.3 El preprocesamiento, que no es un detalle

Los datos vienen en $[0, 1]^2$. **Entrenar sobre ellos tal cual no funciona:**

preprocesamiento	aciertos test	errores ult. epoca
crudo $[0, 1]$	63.10%	1499
centrado $(x - 0.5)$	63.10%	1499
centrado y escalado $\times 4$	95.10%	504

El 63,1% es exactamente la proporción de la clase $+1$ en el conjunto de prueba: **la red no aprende nada y contesta siempre la clase mayoritaria**.

PARA LA DEFENSA — por qué centrar no alcanza y escalar sí

Es el recuadro de la sección 4 del apunte 04, visto del otro lado. Ahí el problema era **saturación**: pesos grandes, v grande, $\varphi' \approx 0$, no aprende. Acá es el problema **opuesto**: con entradas en $[-0,5; 0,5]$ y pesos iniciales en $[-0,5; 0,5]$, la salida lineal v queda diminuta, la sigmoide trabaja en su tramo casi recto y la red se comporta como un modelo lineal — que sobre este problema no puede hacer más que votar por la clase mayoritaria. Escalar la entrada le da al producto interno el rango que necesita. Es equivalente a subir b o a permitir pesos más grandes: **son el mismo grado de libertad**, el de la sección 3 del apunte 03.

7.4 El barrido de arquitecturas

Con los datos escalados, tres semillas por configuración, 60 épocas:

ocultas	semilla 0	semilla 1	semilla 2	medio
2	63.10%	63.10%	69.30%	65.17%
3	94.60%	63.10%	63.10%	73.60%
4	95.50%	94.80%	63.10%	84.47%
6	95.40%	95.40%	63.10%	84.63%
8	96.30%	95.00%	93.40%	94.90%
12	94.30%	93.60%	95.80%	94.57%

IDEA DE FONDO — la tabla es la teoría, medida

Leela contra el paso 3 del apunte 04 §1: - **2 ocultas nunca funciona**. No es mala suerte: con dos semiplanos **no se puede cerrar una región**. Es una imposibilidad geométrica, igual que el XOR con una recta. - **3 a 6 funciona a veces**. Ya se puede cerrar el polígono, pero se está en el mínimo justo y la convergencia depende de la inicialización. - **8 en adelante funciona siempre**, y el rendimiento se planta en $\sim 95\%$. Pasar de 8 a 12 no mejora: el techo ya no lo pone la arquitectura. Y el techo medido ($\sim 96\%$) contra el círculo ideal ($99,5\%$) es la diferencia entre **un polígono y una circunferencia**. Se ve en el gráfico.

7.5 El código

```
FACTOR_DE_ESCALA = 4.0
CENTRO_DE_LOS_DATOS = 0.5
```

```
def normalizar(entradas):
    return (entradas - CENTRO_DE_LOS_DATOS) * FACTOR_DE_ESCALA
```

```
entradas_entrenamiento, deseadas_entrenamiento = cargar_patrones(
    DIRECTORIO_DATASET / "concent_trn.csv")
entradas_prueba, deseadas_prueba = cargar_patrones(
    DIRECTORIO_DATASET / "concent_tst.csv")
```

```
red_concent = PerceptronMulticapa(
    neuronas_por_capa=[2, 8, 1],
    tasa_aprendizaje=0.1,
    semilla=0,
)
red_concent.entrenar(
    normalizar(entradas_entrenamiento),
    deseadas_entrenamiento,
    maximo_epocas=60,
    tolerancia=0.1,
)
```

```
porcentaje_aciertos, cantidad_errores = red_concent.probar(
    normalizar(entradas_prueba), deseadas_prueba)
```

Y para el gráfico que pide la consigna: se evalúa la red sobre una grilla que cubre el plano, se pinta la región según el signo de la salida, y encima se dibujan los patrones de prueba con el color de la clase que la red les asignó.

```
resolucion = 320
malla_x1, malla_x2 = np.meshgrid(np.linspace(0, 1, resolucion),
                                   np.linspace(0, 1, resolucion))
puntos_malla = np.column_stack([malla_x1.ravel(), malla_x2.ravel()])
region = np.where(red_concent.predecir(normalizar(puntos_malla))[:, 0] >= 0, 1.0, -1.0)
region = region.reshape(malla_x1.shape)
```

```
predicciones = np.where(
    red_concent.predecir(normalizar(entradas_prueba))[:, 0] >= 0, 1.0, -1.0)
acertados = predicciones == deseadas_prueba
```

(el script completo con el armado de la figura está en `imagenes/ejercicio2_concent.py`)

7.6 El resultado

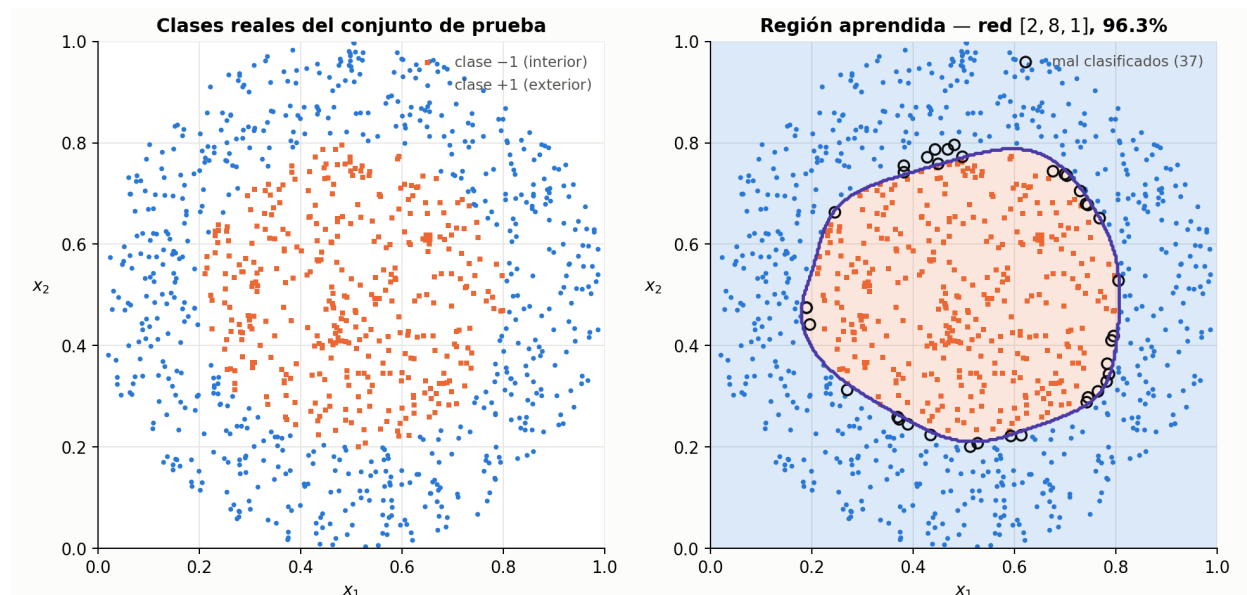


Figura 1: Izquierda, las clases reales. Derecha, la región que aprendió la red: un polígono redondeado de ocho lados, uno por neurona oculta. Los círculos negros son los 37 patrones mal clasificados, todos sobre el borde.

Mirá la forma de la frontera. No es una circunferencia: es un **polígono de lados apenas curvos**, y tiene tantos lados como neuronas ocultas. Es la figura 07 de regiones de decisión, pero aprendida en vez de dibujada.

Y los 37 errores están **todos sobre el borde**, ninguno adentro ni lejos. Eso dice que la red no se equivocó de forma: se equivocó por la resolución del polígono contra el círculo real.

Claves de la sección 7

Clave	Qué tenés que poder responder
El argumento	Los tres pasos, aplicados a las concéntricas
Por qué 2 no anda	Que es imposibilidad geométrica, no mala suerte
El preprocesamiento	Por qué sin escalar da la clase mayoritaria
El techo	Por qué se planta en $\sim 95\%$ y qué lo explica
La figura	Por qué la frontera es un polígono y cuántos lados tiene

8. Lo que todavía no está

El **ejercicio 3** (`iris81`) sigue como `# TODO`. Dos cosas cambian respecto de lo anterior:

- **Tres salidas en vez de una.** Las especies vienen codificadas $[-1, -1, 1]$, $[-1, 1, -1]$ y $[1, -1, -1]$, así que la capa de salida tiene tres neuronas. `cargar_patrones` ya tiene el parámetro `n_salidas` para eso, y aplica lo del apunte 04 §2 sobre por qué con más de una salida **ya no estás limitado a dos clases**.
- **probar hay que cambiarlo.** Como está, decide por el signo de una única salida. Con tres neuronas la regla de decisión pasa a ser *la clase de la neurona con mayor salida*.

Además la consigna pide **variar la tasa de aprendizaje** y graficar las curvas de error cuadrático y de clasificación por época. `entrenar` ya devuelve las dos listas; falta el barrido y los gráficos.